

NS-BRKGA para o Multi-Objective Time- and Machine-Dependent Scheduling Problem

Luis H. P. Mendes Fábio L. Usberti Mário C. San Felice Carlos E. de Andrade

Institute of Computing – University of Campinas (UNICAMP)
Department of Computing – Federal University of São Carlos (UFSCar)
AT&T Labs Research

`luis.mendes@ic.unicamp.br`

LVIII Simpósio Brasileiro de Pesquisa Operacional — SBPO 2026

Motivation & Context

Scheduling FOTA Updates for Connected Vehicles

Firmware-Over-The-Air (FOTA) updates must be delivered to **millions of connected vehicles** through a cellular network.

Key challenges:

- Vehicles are only **available in certain time periods** and **cell sectors**.
- Each cell sector has **time-varying capacity**.
- A global **bandwidth cap** limits total throughput per period.
- The scheduling must be done at **large scale** (up to 10,000 jobs or machines).

From TMDSP to MOTMDSP

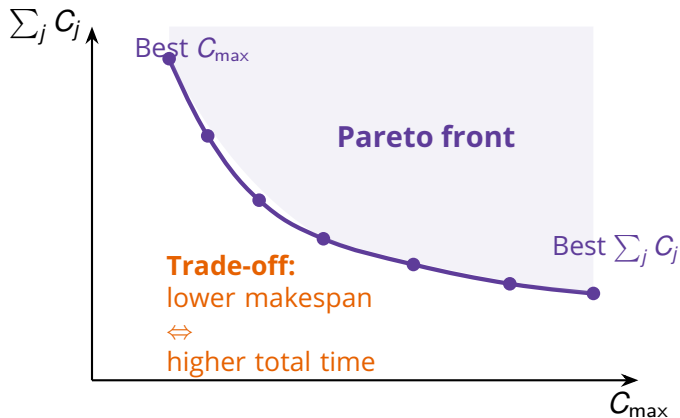
The **Time- and Machine-Dependent Scheduling Problem (TMDSP)** minimizes only $\sum_j C_j$ (total completion time).

Problem: minimizing $\sum_j C_j$ does *not* necessarily minimize $C_{\max}$.

- The correlation between $\sum_j C_j$ and $C_{\max}$ is **weak**.
- An optimal $\sum_j C_j$ schedule may leave some jobs finishing **much later** than others.

⇒ We propose the **Multi-Objective TMDSP (MOTMDSP)**: minimize both $C_{\max}$ and $\sum_j C_j$ **simultaneously**.

Multi-Objective Trade-Off



Decision makers choose schedules along the front based on operational priorities.

Problem Definition

MOTMDSP – Key Elements

Sets:

- J : jobs (vehicles to update)
- M : machines (cell sectors)
- T : discrete time periods

Parameters:

- c_{mt} : capacity of machine m at period t
- η_{mt} : production rate (data per unit)
- q_{jmt} : resource consumption of job j
- d_j : total demand of job j

Constraints:

- Each job assigned to ≤ 1 machine per period
- Machine capacity c_{mt} respected
- Global bandwidth cap $r_{\max}$
- At most $k_{\max}$ active jobs per period
- Minimum resource usage $r_{\min}$

Decision: assign each job to machines and periods until its demand d_j is fully served.

Objective Functions

Minimize simultaneously:

Makespan

$$\min C_{\max} = \max_{j \in J} C_j$$

The **last job's completion time**.

Ensures no job finishes too late.

Total Completion Time

$$\min \sum_{j \in J} C_j$$

The **sum of all completion times**.

Maximizes global throughput.

These two objectives are **conflicting**: improving one typically worsens the other.

Proposed Method: NS-BRKGA

Why NS-BRKGA?

NS-BRKGA hybridizes two powerful frameworks:

From BRKGA-MP-IPR:

- Random-key chromosome representation
- Multi-parent biased crossover
- Implicit path-relinking
- Shaking & partial reset

From NSGA-II:

- Non-dominated sorting
- Crowding distance diversity
- Pareto-based elite selection

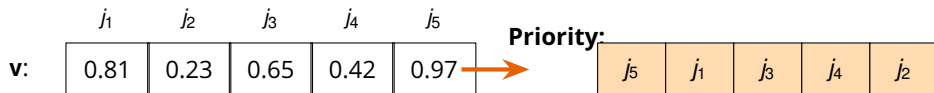
Key advantage: the evolutionary engine is **problem-independent**. All MOTMDSP-specific logic resides in a **specialized decoder**.

Random-Key Representation

A chromosome is a vector of real-valued keys:

$$\mathbf{v} = (v_1, v_2, \dots, v_{|J|}), \quad v_j \in [0, 1).$$

Decoding: sort jobs in **decreasing order** of their keys $\Rightarrow$ this defines a **priority order** for scheduling.



Small perturbations in keys $\Rightarrow$ small changes in job order. Enables **smooth exploration** of scheduling permutations.

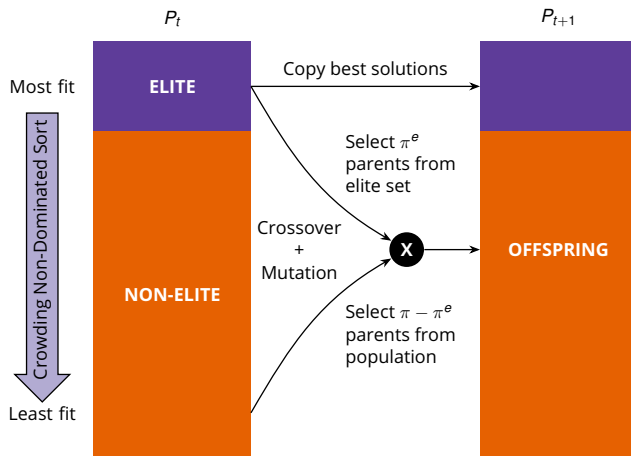
Specialized Decoder

Given a chromosome $\mathbf{v}$, the decoder builds a **feasible left-shifted schedule**:

- 1 **Sort** jobs by decreasing key value (priority order).
- 2 For each job in priority order:
 - Assign to the **earliest feasible machine-period** intervals.
 - Respect all constraints (c_{mt} , $r_{\max}$, $k_{\max}$, $r_{\min}$).
 - Continue until demand d_j is fully served.
- 3 Compute C_j for each job, then output $\mathbf{F} = (C_{\max}, \sum_j C_j)$.

The decoder **always produces a feasible schedule**: no repair or penalty mechanism is needed.

Evolutionary Cycle



Additional NS-BRKGA Features

- **Biased multi-parent crossover**
 $\pi = 3$ parents, bias towards elite.
- **Polynomial mutation**
Small perturbations of random keys.
- **Genotypic diversity**
at elite selection step.
- **Multiple populations**
with periodic migration.
- **Implicit Path-Relinking**
Interpolates keys between elite solutions.
- **Shaking**
Strong mutation on stagnation.
- **Partial reset**
Injects fresh random individuals.

Computational Experiments

Experimental Setup

- **96 instances** from the realistic FOTA/TMDSP benchmark:
 - `common`: vehicles appear in many sectors/periods.
 - `rare`: vehicles appear in few sectors/periods.
 - Up to **10,000 jobs** or **10,000 machines**.
- **5 baseline algorithms** (via PaGMO2, default parameters): NSGA-II, NSPSO, MOEA/D-DE, MHACO, IHS.
- **5 independent runs** per algorithm and instance.
- **Time budget**: 900 seconds (15 min) per run.
- CPU: Intel Xeon E3-1230 V2 @ 3.30 GHz, 32 GB RAM, single core.

Performance Indicators

Two complementary quality indicators:

① **Normalized Modified Inverted Generational Distance** ($NIGD^+$)

- Measures **convergence** to the reference front.
- **Lower is better.**

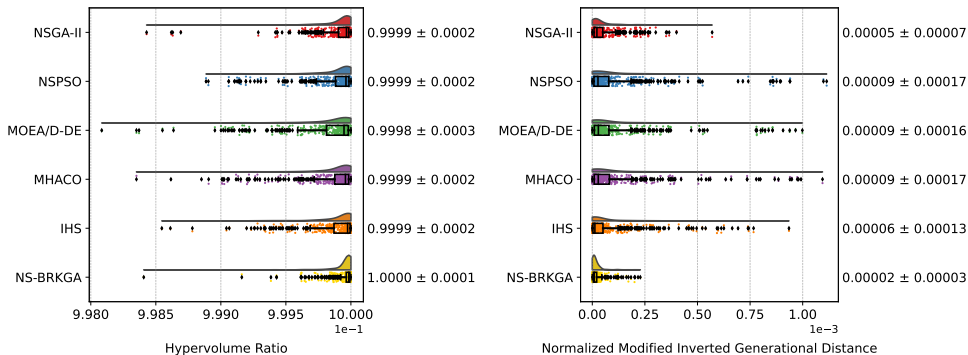
② **Hypervolume Ratio** (HVR)

- Measures both **convergence and diversity**.
- **Higher is better.**

Reference fronts: aggregation of all non-dominated solutions from **all algorithms across all runs**.

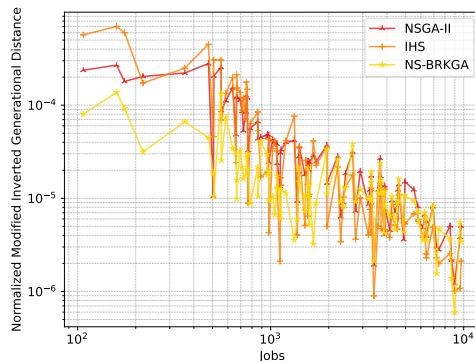
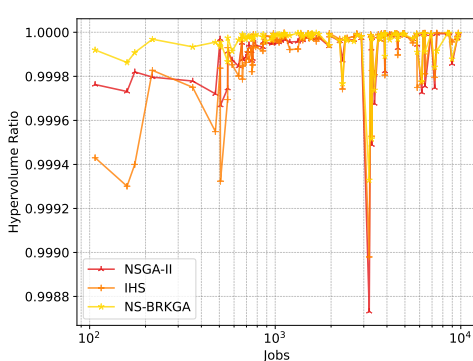
Results & Analysis

Overall Performance



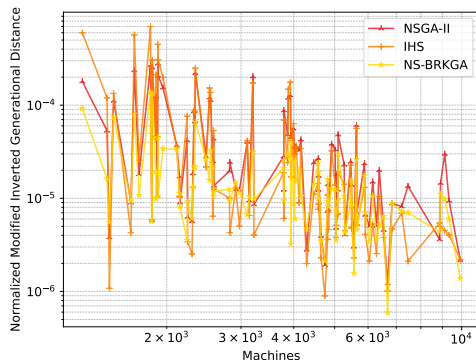
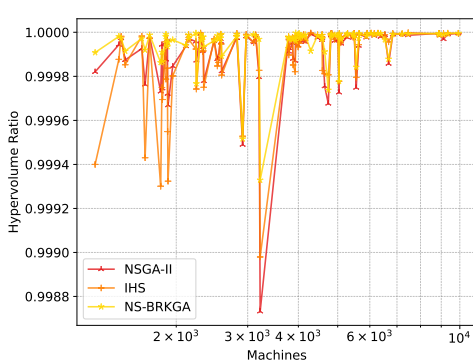
- All algorithms achieve $HVR \geq 0.9998$ — near saturation.
- $NIGD^+$ provides sharper differentiation: NS-BRKGA achieves the **lowest** mean $NIGD^+$ (2×10^{-5} , vs. $5\text{--}9 \times 10^{-5}$ for baselines).

Scalability – Number of Jobs



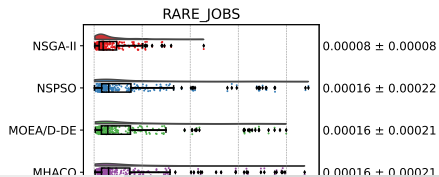
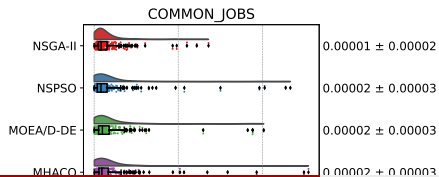
- *HVR* remains near 1.0 and *NIGD*⁺ stays low across all job counts.
- **No degradation** as the number of jobs increases.
- NS-BRKGa consistently among the best curves.

Scalability – Number of Machines



- Similar stability as the number of machines grows.
- All methods maintain strong trade-offs at scale.
- NS-BRKGa appears among the top-performing curves.

Instance Type: common VS. rare



Key Observations: common VS. rare

common instances:

- All methods achieve nearly identical performance.
- $HVR \approx 1$ and $NIGD^+ \sim 10^{-5}$ for all algorithms.

rare instances:

- Convergence is **more challenging** for all algorithms.
- NS-BRKGA stands out with:
 - Best HVR (0.99997).
 - Substantially lower $NIGD^+$ (0.00003 vs. 0.00008–0.00016).
- Indicates greater **robustness** under irregular availability patterns.

Conclusion

Conclusion

- Proposed the **MOTMDSP**: a bi-objective extension of TMDSP that captures the trade-off between makespan and total completion time.
- Developed an **NS-BRKGA** approach with a specialized greedy decoder.
- Extensive experiments on **96 large-scale FOTA instances** show: NS-BRKGA is consistently competitive or best on both *HVR* and *NIGD*⁺.
- Especially strong on **rare instances**: greater robustness under atypical availability.
- **Practical impact**: diverse Pareto-optimal schedules that help decision makers balance throughput vs. last completion time.

Thank you!

Questions?

`luis.mendes@ic.unicamp.br`